

CODEALPHA

Cyber Security Internship

TASK 1

Basic Network Sniffer

By: Ayat Zahra

Capturing, Parsing & Analyzing Live Network Traffic with Python

Prepared as part of the CodeAlpha Cyber Security Internship Program

www.codealpha.tech

1. Objective

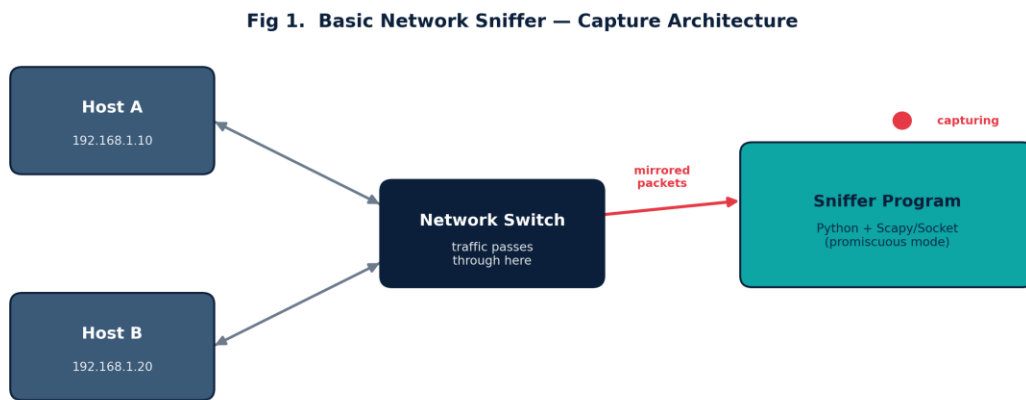
Build a lightweight, working network sniffer in Python that captures live traffic passing through a network interface, decodes the header fields of each packet, and displays useful, human-readable details — source and destination IP addresses, transport-layer protocol, ports, and payload preview. This task builds foundational intuition for how data actually moves across a network and how security tools observe that traffic.

Learning Goals

- Understand how packets are structured and encapsulated (Ethernet → IP → TCP/UDP → payload).
- Learn to capture raw traffic using the Scapy and socket libraries in Python.
- Practice parsing and interpreting protocol headers programmatically.
- Build comfort with the tools (Wireshark, Scapy) used throughout the cybersecurity field.

2. How Packet Sniffing Works

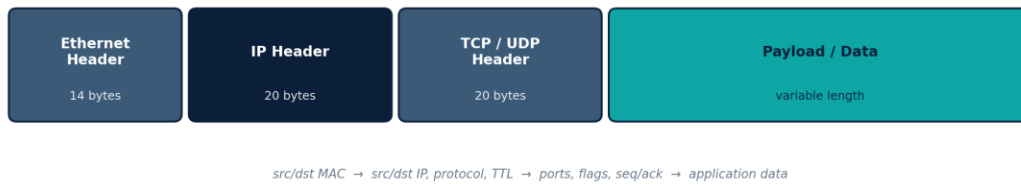
Every device on a network sends and receives data in discrete units called packets (or frames, at the link layer). A sniffer puts a network interface into promiscuous mode, allowing it to read every frame that reaches the interface — not just frames addressed to the host itself. The captured frames are then parsed layer by layer to expose the information each protocol header carries.



The sniffer places its network interface in promiscuous mode to observe traffic traversing the local segment, then parses each captured frame.

Figure 1 — A sniffer process observing traffic as it passes through a shared network segment.

Each captured unit is layered like an onion: the outer Ethernet header identifies hardware (MAC) addresses, the IP header identifies logical (IP) addresses and routing information, the TCP/UDP header identifies the specific application ports and connection state, and finally the payload carries the actual application data.

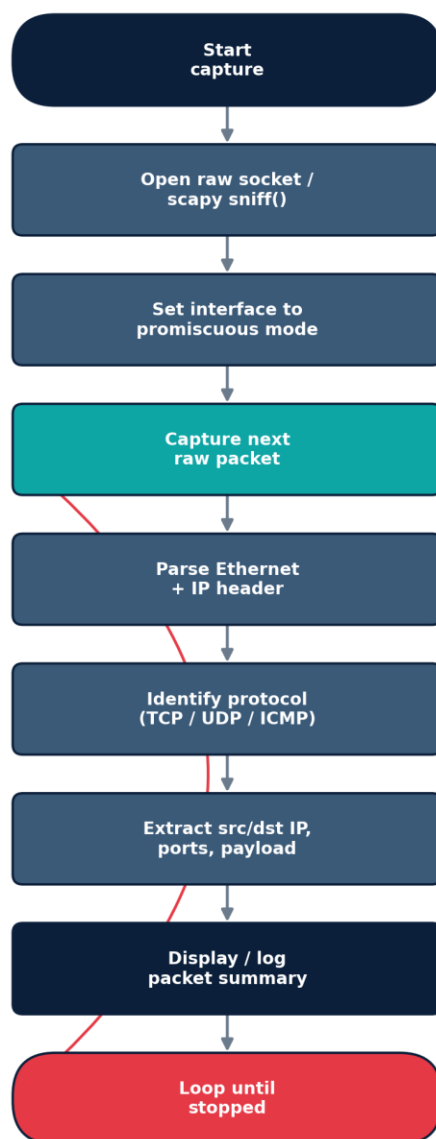
Fig 2. Anatomy of a Captured Packet*Figure 2 — Layered structure of a typical captured packet.*

3. Tools & Libraries

Library	Role in this Task
scapy	High-level packet capture, dissection, and crafting — the primary tool used below.
socket (AF_PACKET)	Low-level raw-socket capture; a dependency-free fallback on Linux.
struct	Unpacks raw byte headers into Python fields for the manual/socket version.
datetime	Timestamps each captured packet for the log/output.

4. Program Workflow

The sniffer runs a simple capture loop: open the interface, read the next packet, parse its headers, extract the fields we care about, print a summary line, and repeat until the user stops the program.

Fig 3. Sniffer Program Flow*Figure 3 — Control flow of the sniffer program.*

5. Source Code — Scapy Version (Recommended)

This is the primary implementation. Scapy handles interface access and header parsing internally, which keeps the code short and readable while still exposing every field we need.

```
#!/usr/bin/env python3
"""
Basic Network Sniffer
CodeAlpha Cyber Security Internship -- Task 1
"""

from scapy.all import sniff, IP, TCP, UDP, ICMP, Raw
from datetime import datetime
```

```

packet_count = 0

def process_packet(packet):
    """Called once for every packet the sniffer captures."""
    global packet_count
    packet_count += 1

    if not packet.haslayer(IP):
        return # skip non-IP traffic (e.g. ARP)

    ip_layer = packet[IP]
    src_ip, dst_ip = ip_layer.src, ip_layer.dst
    proto_name = "OTHER"
    ports = ""

    if packet.haslayer(TCP):
        proto_name = "TCP"
        ports = f"{packet[TCP].sport} -> {packet[TCP].dport}"
    elif packet.haslayer(UDP):
        proto_name = "UDP"
        ports = f"{packet[UDP].sport} -> {packet[UDP].dport}"
    elif packet.haslayer(ICMP):
        proto_name = "ICMP"

    timestamp = datetime.now().strftime("%H:%M:%S")
    print(f"[{timestamp}] #{packet_count:04d} {proto_name:5s} "
          f"{src_ip:15s} -> {dst_ip:15s} {ports}")

    if packet.haslayer(Raw):
        payload = bytes(packet[Raw].load)
        preview = payload[:48]
        print(f"    payload ({len(payload)}B): {preview}")

def main():
    print("Starting network sniffer -- press Ctrl+C to stop.\n")
    try:
        # store=False keeps memory flat during long captures
        sniff(prn=process_packet, store=False)
    except KeyboardInterrupt:
        print(f"\nCapture stopped. Total packets analysed: {packet_count}")
    except PermissionError:
        print("Run this script with administrator / root privileges.")

if __name__ == "__main__":
    main()

```

Optional Filters

Scapy accepts BPF-style filters so you can narrow the capture, e.g. `sniff(filter="tcp port 80", prn=process_packet)` to isolate HTTP traffic, or `filter="icmp"` to watch only ping traffic.

6. Source Code — Raw Socket Version (No External Libraries)

Where Scapy cannot be installed, the same behavior can be built directly on Python's socket module using a raw AF_PACKET socket (Linux only). This version manually unpacks the Ethernet and IP headers with struct.

```
#!/usr/bin/env python3
"""
Basic Network Sniffer -- raw socket fallback (Linux)
"""

import socket
import struct

def format_mac(raw_bytes):
    return ":".join(f"{b:02x}" for b in raw_bytes)

def parse_ethernet(data):
    dst, src, proto = struct.unpack("! 6s 6s H", data[:14])
    return format_mac(dst), format_mac(src), socket.htons(proto), data[14:]

def parse_ip(data):
    version_ihl = data[0]
    ihl = (version_ihl & 0x0F) * 4
    ttl, proto, src, dst = struct.unpack("! 8x B B 2x 4s 4s", data[:20])
    return proto, socket.inet_ntoa(src), socket.inet_ntoa(dst), data[ihl:]

def main():
    raw_socket = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.ntohs(3))
    print("Sniffing on all interfaces -- press Ctrl+C to stop.\n")
    count = 0
    try:
        while True:
            raw_data, _ = raw_socket.recvfrom(65535)
            dst_mac, src_mac, eth_proto, payload = parse_ethernet(raw_data)
            if eth_proto != 8: # 0x0800 = IPv4
                continue
            proto, src_ip, dst_ip, rest = parse_ip(payload)
            count += 1
            print(f"#{count:04d} {src_ip:15s} -> {dst_ip:15s} proto={proto}")
    except KeyboardInterrupt:
        print(f"\nCapture stopped. Total packets: {count}")

if __name__ == "__main__":
    main()
```

7. Code Walkthrough

- `sniff(prn=process_packet)` hands every captured packet to a callback function instead of blocking the main thread manually.

- `haslayer()` checks let the code branch cleanly between TCP, UDP and ICMP without deep manual parsing.
- Fields such as `.src`, `.dst`, `.sport` and `.dport` come from Scapy's automatic header dissection.
- The raw Raw layer, when present, holds whatever bytes sit above the transport header — typically application data.
- The socket version reproduces the same idea manually: `struct.unpack()` reads fixed-width binary fields directly out of the byte string using a format specifier.
- Both versions print a running counter and timestamp so captured traffic can be reviewed chronologically.

8. Sample Output

A short capture on a typical home network, while browsing a website and pinging a host, produces output similar to the table below (payload bytes truncated for readability).

#	Time	Proto	Source	Destination	Ports
0001	10:42:01	TCP	192.168.1.10	142.250.72.14	51422 → 443
0002	10:42:01	TCP	142.250.72.14	192.168.1.10	443 → 51422
0003	10:42:03	UDP	192.168.1.10	192.168.1.1	58122 → 53
0004	10:42:03	UDP	192.168.1.1	192.168.1.10	53 → 58122
0005	10:42:07	ICMP	192.168.1.10	8.8.8.8	—

9. Legal & Ethical Note

Only capture traffic on networks you own or are explicitly authorised to test — for example your own home lab or a virtual machine sandbox. Intercepting traffic on networks without permission is illegal in most jurisdictions and violates the CodeAlpha internship code of conduct. Use this project strictly for learning and authorised testing.

10. Conclusion & Learning Outcomes

Completing this task demonstrates the ability to capture live network traffic, decode multi-layer protocol headers, and present the results in a readable form — core skills that underpin intrusion detection, traffic analysis, and network forensics work in cybersecurity.

- Practical understanding of how the Ethernet / IP / TCP / UDP stack fits together on the wire.
- Hands-on experience with Scapy for rapid packet analysis tooling.
- A reusable raw-socket fallback for environments without third-party libraries.
- A foundation for later tasks, e.g. building simple intrusion-detection rules on top of this capture loop.

References

Scapy documentation — scapy.readthedocs.io • Python socket module — docs.python.org/3/library/socket.html